

General and Robust Loss Function for Uncertainty Estimation

Deebul Nair¹, Nandhini Mathivanan², Miguel A. Olivares-Mendez¹ and Nico Hochgeschwender³

Abstract—The negative log likelihood loss function based uncertainty estimation often struggles with outliers in datasets, hindering the performance of deep neural networks (DNNs) in robotics applications. To address this limitation, we present a generalization of negative log likelihood based uncertainty estimation. Our loss function generalizes algorithms built around negative log likelihood loss minimization by introducing a new robustness parameter, which improves performance on basic robotics tasks that involve datasets with outliers. By generalizing the negative log likelihood loss, our approach extends its applicability to a wider range of distributions, including the normal, Laplace, and Cauchy distributions. This flexibility allows the DNN to dynamically adapt its learning process based on the percentage of outliers present in the dataset. Our experiments show that the proposed loss function is significantly more resistant to outliers than existing uncertainty estimation methods, thus improving performance on robotics tasks such as keypoint estimation and antipodal grasping.

I. INTRODUCTION

Robustness to outliers in data is an important factor when choosing a loss function for training deep neural networks (DNN). This is especially true in robotics tasks, where datasets often contain noisy labels and outliers. This requirement also applies to uncertainty estimation methods, which are essential for deploying DNN in safety-critical robotics tasks [1], [2]. Researchers have developed various robust uncertainty estimation methods that can estimate correct uncertainty in the presence of outliers [3], [4]. Currently, roboticists spend considerable time either cleaning datasets or experimenting with different robust loss functions. In this work, we aim to answer the following research question: *When training a regression model on a dataset containing a certain percentage of outliers, how can the model accurately estimate the uncertainty of model's predictions by selecting the appropriate robust loss function?*

Uncertainty estimation in DNN is typically categorized into three main approaches: 1) Bayesian methods [5], [6]; 2) Sampling based methods [7], [8] and 3) Single model methods [9] [10], [11], [3]. While Bayesian and sampling-based methods often deliver state-of-the-art performance [12], [13], they come with significant computational overhead. Bayesian methods require doubling the model size

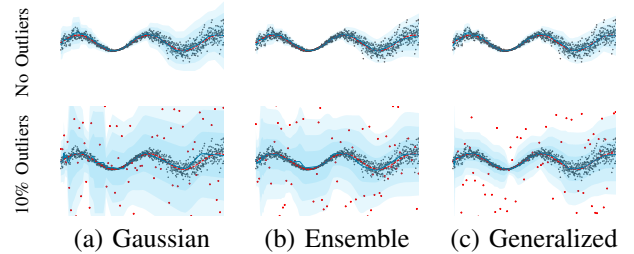


Fig. 1. Illustrative comparison of uncertainty estimation on 1D regression data with outliers. Row 1 shows data with no outliers while row 2 data contains 10% outliers. The blue dots are clean data while red dots are outlier. The shaded blue region is the estimated uncertainty. Both Gaussian and Ensemble methods are more sensitive to outliers, the blue region are more distorted by the outlier data points, while for generalized the blue region has distorted less indicating less impact of outliers.

and inference time [14], while sampling methods increase inference time substantially [7], [8]. These drawbacks make them impractical for robotics applications that demand lean models and fast inference times. Single-model uncertainty estimation, on the other hand, can assess uncertainty with a single pass through the model and only a minimal increase in model size [9], [15], [11]. Given its suitability for robotics systems, we focus on single-model uncertainty estimation in this work.

Negative likelihood-based loss functions offer a simple approach to uncertainty estimation in DNN. These methods model the output of the DNN as a statistical distribution, often the normal distribution. The loss function is then calculated as the negative log-likelihood (NLL) of the probability density function (PDF) of the chosen distribution. Studies have shown that using fat-tailed distributions like the Laplace distribution can make the model more robust to outliers in the data [3]. However, selecting the most appropriate distribution for a specific dataset still remains a manual process.

In this paper, we present a general uncertainty estimation loss function that is a super-set of common robust uncertainty estimation loss functions. Our loss function is the negative log likelihood of the generalized normal distribution [16]. Using the negative log-likelihood of the generalized normal distribution as our loss function, we demonstrate that by minimizing the loss, any gradient-based optimization can automatically adjust the robustness of the loss without needing manual intervention. This flexible form of our loss function is especially useful for models that produce high

² Institute for Artificial Intelligence and Autonomous Systems, Bonn-Rhein-Sieg University of Applied Sciences, Grantham-Allee 20, 53757 Sankt Augustin, Germany. {nandhini.mathivanan@mail.inf.h-brs.de, teena.hassan@h-brs.de}

¹ Space Robotics Research Group (SpaceR), Interdisciplinary Centre for Security, Reliability and Trust (SnT), University of Luxembourg, Luxembourg. miguel.olivaresmendez@uni.lu

³ University of Bremen, Germany. Department of Mathematics and Computer Science nico.hochgeschwender@uni-bremen.de

dimensional outputs, like those used for depth estimation and keypoint estimation. The loss function can assign a unique robustness value to each output dimension, allowing the model to adjust the robustness of its loss for each dimension individually. This robustness to outliers of our proposed loss function is demonstrated using a 1D regression dataset containing both heteroskedastic noise and outliers in Figure 1. Uncertainty predicted by Gaussian and Ensemble methods are heavily affected by outliers, while uncertainty predicted by our proposed method degrades the least while maintaining same accuracy.

This paper investigates the use of the generalized normal distribution [16] for maximum likelihood-based uncertainty estimation. Our work makes the following contributions:

- 1) We propose a new loss function that adapts between multiple robust loss functions, based on the outliers in the data and thus improves the robustness of maximum likelihood-based uncertainty estimation methods;
- 2) We evaluate the estimated uncertainty on different regression benchmarks and also on a complex vision regression tasks of keypoint estimation; and
- 3) We demonstrate and evaluate the use of estimated uncertainty by conducting a robotic experiment for grasp selection for antipodal grasp.

II. ROBUST UNCERTAINTY ESTIMATION

In a regression learning problem, we work with a training dataset D that consists of pairs (x, y) drawn from a joint distribution $D(X, Y)$. Each pair contains an input x from the input set X and a corresponding label y from the label set Y . We use a neural network $f(\theta)$, where θ represents the network's parameters. Our goal is to train these parameters by minimizing an empirical loss function l . This can be expressed as:

$$\hat{\theta} = \min_{\theta} R(\theta); \quad R(\theta) = \frac{E}{\langle x, y \rangle \in D} [\ell(x, y, \theta)]$$

, where $R(\theta)$ is the expected loss over the dataset D . In a standard regression scenario (without considering robustness or outliers), we can use the sum of squared residuals as our loss function:

$$\ell(x, y, \theta) = \frac{1}{2}(y - f_{\theta}(x))^2$$

The square of residuals is a reformulation of the negative log likelihood of a normal distribution. This approach provides a point estimate, essentially estimating the mean (μ) of a normal probability distribution. However, it's limited in its ability to provide uncertainty estimation. A straightforward enhancement to the basic regression model is to estimate the complete conditional probability distribution. This can be achieved by predicting both the mean (μ) and the variance (σ^2) of the distribution, assuming it follows a Gaussian (normal) distribution. This approach, as described by [9], provides more information about the uncertainty of predictions.

Although the NLL of a normal distribution allows us to estimate both the value and the uncertainty in the prediction,

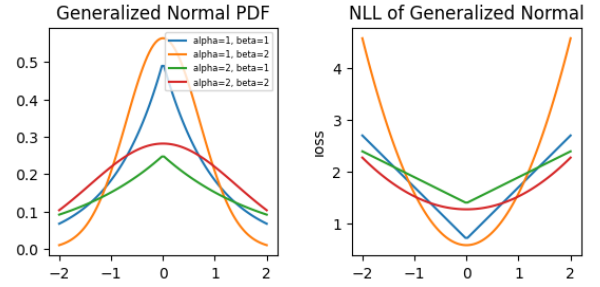


Fig. 2. Probability density function (PDF) (left) and negative log likelihood (NLL) (right) of Generalized normal distribution

it suffers from a lack of robustness. When the training data contains even a small number of outliers, it can lead to significant shifts in both the predicted values and their associated uncertainties. This sensitivity to outliers means that these loss functions have a low tolerance for outlier data points. In statistical terms, these loss functions have a high breakaway point [17].

For enhancing the robustness of NLL-based uncertainty estimation against outliers, various loss functions have been proposed utilizing fat-tailed distributions [3]. While these approaches have demonstrated effectiveness in mitigating the impact of outliers, the selection of the appropriate distribution hyperparameter often remains a manual process, relying on empirical experimentation.

In this work, we introduce a novel loss function that adaptively selects the suitable distribution based on the presence of outliers within the dataset. Our proposed loss function is grounded in the generalized normal distribution, a versatile distribution capable of representing different data distributions, including normal, Laplace, and Cauchy distributions. This flexibility enables our loss function to effectively model data with varying levels of tail behavior.

A. Generalized Normal Loss Function

The probability density function (PDF) of the generalized normal distribution is given by:

$$p(r; \alpha, \beta) = \frac{\beta}{2\alpha\Gamma(1/\beta)} \exp \left[- \left(\frac{|r|}{\alpha} \right)^\beta \right] \quad (1)$$

where: r is the random variable. α is the scale parameter, controlling the spread of the distribution and β is the shape parameter, controlling the tail behavior of the distribution. The parameters α and β both are positive real numbers. The NLL of the distribution is the loss function used for training. For the generalized normal distribution, the NLL can be derived from the PDF as follows:

$$NLL(\alpha, \beta; r) = - \sum_{i=1}^N \log f(r_i; \alpha, \beta) \quad (2)$$

where: N is the number of data points r_i is the i -th error or residual. Figure 2 plots both the PDF and the NLL for

different values of α and β . Substituting the PDF into the NLL equation, we get:

$$NLL(\alpha, \beta; r) = \underbrace{\left(\frac{|r|}{\alpha}\right)^\beta}_{\text{term 1}} - \underbrace{\log \beta}_{\text{term 2}} + \underbrace{\log \left(2\alpha\Gamma\left(\frac{1}{\beta}\right)\right)}_{\text{term 3}} \quad (3)$$

The loss function consist of 3 components : 1) term 1: This term measures the magnitude of the residual (error) between a predicted value and an actual value. The parameters α and β control the shape and scale of this term. 2) term 2 : These terms are regularization terms that help prevent over-fitting. They penalize large values of α and small values of β , which can lead to unstable models. 3) term 3: This term involves the gamma function and it ensures that the loss function is well-defined for all values of α and β . The proposed loss function has many properties that makes it suitable for gradient based optimization.

1) *Differentiability*: The loss function is differentiable almost everywhere. To show this, we can compute its partial derivatives with respect to x , α , and β . Partial derivative with respect to r , for $r \neq 0$:

$$\frac{\partial L}{\partial r} = \beta \frac{|r|^{\beta-1}}{\alpha^\beta} \cdot \text{sgn}(r)$$

for $r = 0$:

$$\frac{\partial L}{\partial r} = 0$$

. the partial derivative with respect to α

$$\frac{\partial L}{\partial \alpha} = -\beta \frac{|r|^\beta}{\alpha^{\beta+1}} + \frac{2}{\alpha}$$

and the partial derivative with respect to β

$$\frac{\partial L}{\partial \beta} = \frac{|r|^\beta}{\alpha^\beta} \log \left(\frac{|r|}{\alpha} \right) - \frac{1}{\beta} + \frac{\Gamma'(1/\beta)}{\Gamma(1/\beta)}$$

From these expressions, we can see that the partial derivatives are continuous and defined for all r , α , and β except for $r = 0$. Therefore, the loss function is differentiable almost everywhere. This ensures that the gradient can be computed at most points, allowing the optimization algorithm to make progress.

2) *Unimodality*: While the loss function is not strictly convex, it is generally unimodal. This means that there is a single global minimum. To show this, we can analyze the second-order partial derivatives: Second-order partial derivative with respect to r :

$$\frac{\partial^2 L}{\partial r^2} = \beta(\beta-1) \frac{|r|^{\beta-2}}{\alpha^\beta}$$

From this expression we can see that the loss function is unimodal and thus has a single global minimum, making it easier for the optimization algorithm to converge to the optimal solution.

3) *Boundedness*: The loss function is bounded from below. The term $\log \Gamma(1/\beta)$ is bounded from below, and the remaining terms are non-negative. Therefore, the loss function cannot be arbitrarily negative.

Finally, the parameters α and β can be interpreted as scale and shape parameters, respectively, providing insights into the behavior of the loss function.

B. Different Uncertainty Estimation Loss Functions

The generalized normal loss function consist of different NLL loss functions based on the β parameter.

1) *Normal loss function*: : When $\beta = 2$

$$NLL(\alpha, \beta = 2; r) = \frac{r^2}{2\alpha^2} + 2 \log(\alpha)$$

2) *Laplace loss function*: : When $\beta = 1$

$$NLL(\alpha, \beta = 1; r) = \frac{|r|}{\alpha} + 2 \log(\alpha)$$

3) *Cauchy loss function*: : When $\beta \rightarrow \infty$

$$NLL(\alpha, \beta \rightarrow \infty; r) = \log \left(1 + \frac{r^2}{\alpha^2} \right)$$

The generalized normal loss function provides a flexible framework for modeling different types of errors and can be adapted to specific needs by adjusting the parameters α and β .

III. RELATED WORK

Uncertainty Estimation: In [9] a novel approach was introduced for uncertainty estimation for neural networks by modeling outputs as Gaussian distributions and employing a negative log likelihood of the distribution as the loss function. This method demonstrated the neural network's ability to learn both predictions and uncertainty. In [14], DNN uncertainty estimation was achieved by representing network weights with distributions. [7] proposed a sampling based techniques using dropouts and multiple forward passes and capture the differences in the different passes as uncertainty. Ensemble methods, as exemplified by [8], [12] and [13] currently dominate state-of-the-art uncertainty estimation. These methods leverage multiple models within a single framework to provide robust uncertainty estimates. Despite their effectiveness, ensemble methods generally necessitate multiple forward passes, limiting their applicability in real-time or resource-constrained scenarios.

Single Inference Uncertainty Estimation: Several methods can estimate uncertainty using a single inference. These include: 1) Replacing the loss function with NLL of statistical distribution [15], [11], [10], [3], 2) Computing the closed-form posterior for the output layer [18], [19], 3) Changing the output layer [20], [21], 4) Using spectral normalization [22] and 5) Applying a two-sided gradient penalty [23]

	Ensemble	Evidential	Gaussian	Laplace	Ours
boston	1.24	1.55	1.60	1.29	1.23
concrete	1.45	570.23	1.89	1.62	1.42
energy	0.70	51.54	0.88	0.84	0.65
kin8nm	1.25	882.00	1.69	1.43	1.19
naval	0.95	640.84	1.11	1.25	1.10
power	0.94	599.59	1.34	1.07	0.93
protein	3.01	397.89	4.13	3.48	2.91
wine	3.61	4.52	4.27	3.96	3.17
yacht	0.37	0.33	0.47	0.43	0.32

TABLE I: Interval Score for different uncertainty estimation methods trained on different UCI datasets. Our proposed Generalized NLL loss function consistently outperforms most of the baselines.

Robust Uncertainty Estimation: Robust training in the presence of outliers is a classic statistical problem often addressed using M-estimators [17]. For DNN, [4] proposed a generalized M-estimator loss function that uses the negative log of the density function to improve robustness during training. [24] introduced a Gaussian-uniform mixture model as a loss function that can continuously adapt to outliers in the data. In [3] studied the usage of Laplace NLL as loss function for improving robustness to outliers. In this work we work on a extension of this work which generalizes different fat tailed distribution based NLL loss function thus further improving robustness capability of DNN.

IV. EVALUATION

In this section, we evaluate the performance of our proposed loss function. Our focus is on demonstrating its ability to improve robustness to outliers in the data. While these benchmarks do not aim to represent the state-of-the-art for any specific task, they effectively showcase the capabilities of our loss function in learning uncertainty with outlier-contaminated datasets. For comparison, we consider the following uncertainty estimation methods: **Gaussian**, as introduced in [9], **Ensemble**, corresponding to the deep learning ensemble approach proposed by [8], **Evidential**, utilizing the Normal-inverse-Gamma distribution as suggested by [10], **Laplace**, as described in [3] and **Generalized** method described in this paper. We formulate our experiments to answer the following questions:

- 1) **RQ1:** How does DNN trained using Generalized NLL compare with other uncertainty estimation methods on different regression benchmarks ?
- 2) **RQ2:** How much percentage of outliers can the different methods handle before their prediction and uncertainty deteriorates ?
- 3) **RQ3:** How does the proposed loss function compare for a complex vision task of Key point estimation ?
- 4) **RQ4:** How can the uncertainty predicted by the keypoint estimation DNN be used for improving grasping and how do they compare with each other ?

A. Benchmark Regression Tasks

To answer RQ1, we benchmark our uncertainty estimation method on real-world regression datasets. To evaluate a

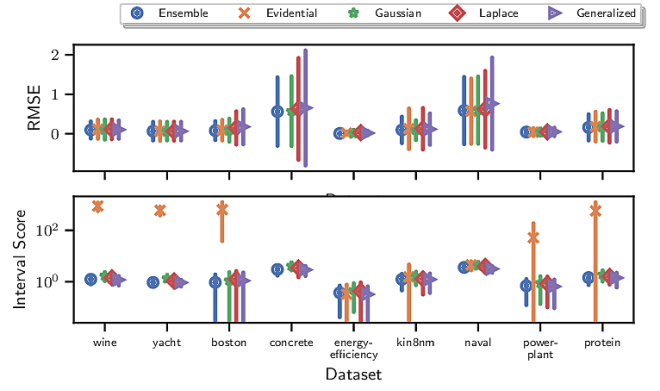


Fig. 3. **Benchmark Regression Tasks:** RMSE and Interval Score box plot for comparison. The RMSE for all the methods are comparable and lie in same region. Interval score which predicts the quality of uncertainty estimation we can see that some methods the score degrades even though their RMSE score is good.

DNN’s ability to estimate uncertainty accurately, we need metrics that separate the quality of uncertainty from the quality of predictions. Scoring rules are a group of metrics that assess uncertainty quality by assigning a numerical score based on both predictions and uncertainty [25]. We use the Interval Score scoring rule, which compares prediction intervals. The Interval Score rewards narrow predictions and penalizes predictions outside the interval. We set α of Interval score to 95% for our experiments. Because the Interval Score compares the 95% quantile prediction interval, it’s comparable across different distributions. We conducted 20 experiments with random training and testing data splits. We used a fully connected neural network with Adam optimization and a learning rate of $5e^{-3}$. Our results show that our proposed method performs the best as compared to other methods in terms of Interval Score as show in Table I.

B. Empirical Breakaway Point

To answer RQ2 about the robustness of different loss functions to outliers, we conducted experiments using a

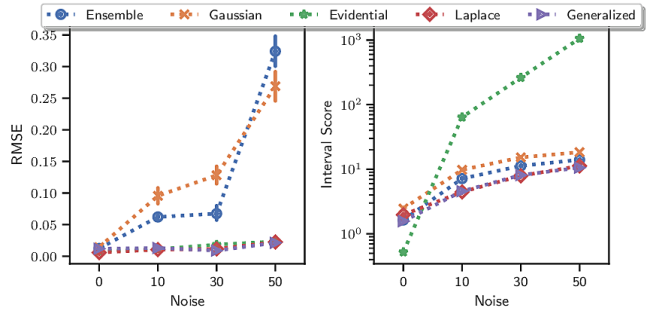


Fig. 4. **Breakaway Point estimation:** The change in RMSE and Interval Score with respect to outlier percentage in the data for 1D regression dataset.

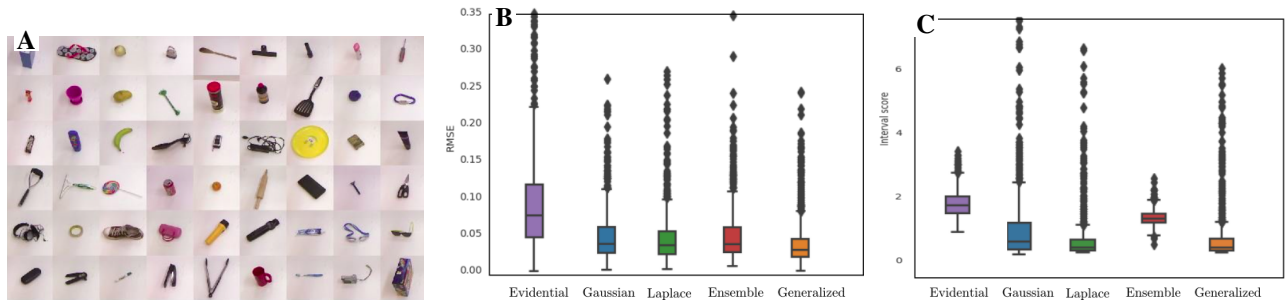


Fig. 5. **Keypoint Estimation** Trained on Cornell Grasp Dataset. (A) Example images from dataset. (B) RMSE comparison plot (C) Interval score plot. Genralized NLL trained model have the lowest RMSE and Interval scores as compared to other uncertainty estimation methods.

1D regression problem with increasing outliers in dataset. We employed a 4-layer neural network and systematically introduced outliers, ranging from 0% to 50% of the total data. The "breakaway point" was determined by observing the threshold at which a method's performance began to deteriorate significantly. This method provided insights into how well each loss function could withstand the presence of outlier data points and maintain accurate predictions and uncertainty. For each experiment, we recorded the RMSE and Interval Score to gauge both prediction accuracy and uncertainty estimation quality.

Our findings as shown in Figure 4 revealed that Gaussian and Ensemble methods exhibited similar levels of robustness, breaking down at approximately 10% outlier contamination. However, Evidential learning, while demonstrating strong performance in uncertainty estimation, was less resilient to outliers in terms of prediction accuracy, breaking down at around 10% as well. We observe how both fat tailed loss function Laplace and Generalized was able to sustain performance even in the presence of 50% outliers. These results underscore the importance of comprehensive evaluation in assessing the robustness of uncertainty estimation models. While a method might excel in one aspect, such as uncertainty estimation, it may be susceptible to outliers in another, such as prediction accuracy. Therefore, it is crucial to consider both factors when selecting a suitable method for a given application.

C. Keypoint Estimation

Having established benchmark comparison results, we now demonstrate the scalability of our generalized NLL learning approach by applying it to the complex, high-dimensional task of keypoint estimation. Keypoint estimation

is a computer vision task that involves identifying and locating specific points of interest within an image. It's essentially a regression problem, where the goal is to predict the continuous x and y coordinates of each keypoint within the image. Its a challenging task due to the high-dimensional nature of the target output, with predictions required at every pixel.

Cornell grasp dataset and Data Augmentation We train our model using the Cornell grasp dataset [26]. The Cornell Grasp Dataset is a collection of real-world images and corresponding annotations for robotic grasping tasks. It contains a diverse range of objects, including everyday items like cups, bottles, tools, and toys. Each image is paired with annotations that specify the most suitable pinch grasp points. The Dataset contains around 1K RGB-D images of 240 different objects. Figure 5 A shows a sample of training images from the dataset. Each image has dimension of 640×480 px and is annotated with multiple potential grasp points, typically ranging from 1 to 10 per image. To address the limited size of the dataset, we implemented data augmentation techniques to increase its diversity during training. Data augmentation methods of random rotations between 0 and 360 degrees were applied, along with random translations in the x and y directions of up to 50 pixels.

Architecture: We utilized a ResNet18 architecture [27] as our architecture. The neural network's output is an 8×2 matrix, representing 8 keypoints with x and y coordinates. For Gaussian and Laplace networks, the output expands to 8×4 , adding two dimensions for the scale parameter. Evidential networks have an 8×8 output, with three additional outputs for each keypoint. Generalized NLL networks output 8×6 . The hyperparameters used are Adam optimization, learning rate of $1e-3$, batch size of 32, and 200 training epochs. *Training Schedule:* The network was trained end-to-end using Adam as the optimizer with a learning rate of $1e-3$, weight decay of 0.001, a momentum factor of 0.9, and momentum enabled. We used a batch size of 32 and trained the models with each loss function for 200 epochs.

Quantitative Results: Table II presents the results on the Cornell grasp dataset. The detailed box plot of the RMSE and Interval scores are plotted in Figure 5 B and C respectively. Our loss function achieves state-of-the-art accuracy com-

Dataset	RMSE	Interval Score
Gaussian	0.0469	1.7960
Laplace	0.0459	0.4348
Ensemble	0.0477	0.9202
Evidential	0.0729	1.1217
Generalized (Ours)	0.0411	0.4331

TABLE II: RMSE and Interval Score Result table for Cornell Grasp dataset



(a)



(b)

Method	Success Rate ($\uparrow$)
Gaussian	36.2 %
Laplace	58.3 %
Cauchy	33.1 %
Evidential	31.9 %
Generalized (ours)	61.5 %

(c)

Fig. 6. Robot grasp benchmark: (a) Experimental setup consisting of a kinova arm with intel realsense camera on its tooltip with one object placed in front of robot for grasping. (b) Novel 10 objects selected for the experiments (c) Table with results of the benchmark

pared to other NLL loss functions. Notably, our Generalized NLL method also outperforms other NLL methods in terms of Interval Score. This shows that our Generalized NLL loss function is able to adjust the shape of the loss function based on the outliers present in the data and is able to adjust.

D. Uncertainty Estimation Based Grasping

In order to study RQ4, we conducted an experiment where the uncertainty can be used on a robotics task and we can benchmark the different loss function on robotics task. We deployed the keypoint estimation DNN models learned in the previous section on a robot arm for object grasping using antipodal grasp. We leverage the predicted uncertainties, into developing a robust grasp selection algorithm. This algorithm chooses the optimal keypoint for physical grasping by selecting a single best keypoint with least uncertainty from the 8 keypoints predicted by the model for each object.

In this experiment, we used a Kinova robot arm equipped with a Robotiq gripper end-effector for object grasping. Ten objects were selected from the Cornell Grasp dataset, as illustrated in Figure . An Intel RealSense camera is available on the robot's end-effector. The objects were arranged on a horizontal surface with a white background in a region with optimal reachability. We use the grasp protocol as recommended in previous research by [28]. Our experiment employs a top-down grasping approach for single objects on a table. Each object is positioned at a predetermined location and orientation relative to the robot. The keypoint estimation algorithm identifies potential grasping points, and the grasp selection algorithm chooses the most suitable keypoint. The grasp planner then converts this keypoint into 3D world coordinates and commands the robot arm to execute the grasp. This process is repeated N times for comprehensive evaluation. As outlined in [28], the results can be evaluated using the metric s/a , where s represents the count of successful grasps and a equals $2 * N$, with N being the number of grasp attempts per object. The factor of 2 accounts for the consideration of two different object orientations: vertical and horizontal. Grasp failures are identified when the gripper

displaces the object, when the object slides or moves during grasping or lifting, or when the grasped item is dropped. This experimental setup is valuable for determining the optimal grasping pose using uncertainty estimation methods and can be evaluated based on the aforementioned criteria.

A final success rate, calculated as the normalized average across all trials and poses is reported in Table III . We observe that the DNN trained with Generalized NLL performs the best with 61.5% success rate in grasping.

V. DISCUSSION

We introduced a generalized loss function that can adapt its shape to outliers in the dataset. This eliminates the need for researchers to manually select a suitable loss function, as the model automatically adjusts to find the appropriate loss function and corresponding distribution. To evaluate the performance of our DNN model trained with this loss function, we tested it on a datasets with varying percentages of outliers. Our results demonstrated that the model trained with generalized NLL loss function is robust to outliers, with minimal impact on both performance and estimated uncertainty. We also benchmarked our loss function on various regression benchmarks and found it to achieve state-of-the-art with respect to uncertainty estimation. Additionally, we applied it to the high-dimensional task of keypoint estimation. Furthermore, we demonstrated the practical utility of the predicted uncertainty in a robotics task: robot grasping.

While our work provides a proof of concept for using this loss function in DNN training, further research is needed to explore its runtime robustness. Adversarial testing and out-of-distribution detection are areas that warrant investigation. We hope that these efforts will contribute to the development of stable uncertainty estimation methods that are resilient to outliers.

ACKNOWLEDGMENT

Deebul Nair gratefully acknowledges the on-going support of the Bonn-Aachen International Center for Information Technology and a PhD scholarship from the Graduate Institute of the Bonn-Rhein-Sieg University.

REFERENCES

- [1] M. Borg, C. Englund, K. Wnuk, B. Duran, C. Levandowski, S. Gao, Y. Tan, H. Kaijser, H. Lönn, and J. Törnqvist, "Safely Entering the Deep: A Review of Verification and Validation for Machine Learning and a Challenge Elicitation in the Automotive Industry," *Journal of Automotive Software Engineering*, vol. 1, no. 1, pp. 1–19, Jan. 2019.
- [2] G. Schwalbe and M. Schels, "A Survey on Methods for the Safety Assurance of Machine Learning Based Systems," in *10th European Congress on Embedded Real Time Software and Systems (ERTS 2020)*, Toulouse, France, Jan. 2020.
- [3] D. S. Nair, N. Hochgeschwender, and M. A. Olivares-Méndez, "Maximum likelihood uncertainty estimation: Robustness to outliers," in *Proceedings of the Workshop on Artificial Intelligence Safety 2022 (SafeAI 2022) co-located with the Thirty-Sixth AAAI Conference on Artificial Intelligence (AAAI2022)*, Virtual, February, 2022, ser. CEUR Workshop Proceedings, G. Pedroza, J. Hernández-Orallo, X. C. Chen, X. Huang, H. Espinoza, M. Castillo-Effen, J. A. McDermid, R. Mallah, and S. Ó. hÉigeartaigh, Eds., vol. 3087. CEUR-WS.org, 2022. [Online]. Available: https://ceur-ws.org/Vol-3087/paper_20.pdf
- [4] J. T. Barron, "A general and adaptive robust loss function," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 4331–4339.
- [5] A. Kendall and Y. Gal, "What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision?" in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [6] M. W. Dusenberry, D. Tran, E. Choi, J. Kemp, J. Nixon, G. Jerfel, K. Heller, and A. M. Dai, "Analyzing the role of model uncertainty for electronic health records," in *Proceedings of the ACM Conference on Health, Inference, and Learning*, 2020, pp. 204–213.
- [7] Y. Gal and Z. Ghahramani, "Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning," p. 10, 2016.
- [8] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [9] D. A. Nix and A. S. Weigend, "Estimating the mean and variance of the target probability distribution," in *Proceedings of 1994 IEEE international conference on neural networks (ICNN'94)*, vol. 1. IEEE, 1994, pp. 55–60.
- [10] A. Amini, W. Schwarting, A. Soleimany, and D. Rus, "Deep Evidential Regression," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 14 927–14 937.
- [11] M. Sensoy, L. M. Kaplan, and M. Kandemir, "Evidential deep learning to quantify classification uncertainty," pp. 3183–3193, 2018.
- [12] M. Dusenberry, G. Jerfel, Y. Wen, Y. Ma, J. Snoek, K. Heller, B. Lakshminarayanan, and D. Tran, "Efficient and scalable bayesian neural nets with rank-1 factors," in *International conference on machine learning*. PMLR, 2020, pp. 2782–2792.
- [13] Y. Wen, D. Tran, and J. Ba, "Batchensemble: an alternative approach to efficient ensemble and lifelong learning," in *International Conference on Learning Representations*, 2020.
- [14] C. Blundell, J. Cornebise, K. Kavukcuoglu, and D. Wierstra, "Weight uncertainty in neural network," in *International Conference on Machine Learning*. PMLR, 2015, pp. 1613–1622.
- [15] A. Malinin, S. Chervontsev, I. Provilkov, and M. Gales, "Regression Prior Networks," *arXiv:2006.11590 [cs, stat]*, Dec. 2020.
- [16] S. Nadarajah, "A generalized normal distribution," *Journal of Applied statistics*, vol. 32, no. 7, pp. 685–694, 2005.
- [17] P. J. Huber, *Robust statistics*. John Wiley & Sons, 2004, vol. 523.
- [18] C. Riquelme, G. Tucker, and J. Snoek, "Deep bayesian bandits showdown: An empirical comparison of bayesian deep networks for thompson sampling," in *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*, 2018.
- [19] J. Snoek, O. Rippel, K. Swersky, R. Kiros, N. Satish, N. Sundaram, M. M. A. Patwary, Prabhat, and R. P. Adams, "Scalable bayesian optimization using deep neural networks," in *Proceedings of the 32nd International Conference on Machine Learning, ICML 2015, Lille, France, 6-11 July 2015*, ser. JMLR Workshop and Conference Proceedings, vol. 37. JMLR.org, 2015, pp. 2171–2180.
- [20] R. Calandra, J. Peters, C. E. Rasmussen, and M. P. Deisenroth, "Manifold gaussian processes for regression," in *2016 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2016, pp. 3338–3345.

- [21] N. Tagasovska and D. Lopez-Paz, “Single-model uncertainties for deep learning,” *Advances in Neural Information Processing Systems*, vol. 32, pp. 6417–6428, 2019.
- [22] J. Z. Liu, Z. Lin, S. Padhy, D. Tran, T. Bedrax-Weiss, and B. Lakshminarayanan, “Simple and principled uncertainty estimation with deterministic deep learning via distance awareness,” in *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020.
- [23] J. Van Amersfoort, L. Smith, Y. W. Teh, and Y. Gal, “Uncertainty estimation using a single deep deterministic neural network,” in *International Conference on Machine Learning*. PMLR, 2020, pp. 9690–9700.
- [24] S. Lathuilière, P. Mesejo, X. Alameda-Pineda, and R. Horaud, “Deepgum: Learning deep robust regression with a gaussian-uniform mixture model,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 202–217.
- [25] T. Gneiting and A. E. Raftery, “Strictly proper scoring rules, prediction, and estimation,” *Journal of the American statistical Association*, vol. 102, no. 477, pp. 359–378, 2007.
- [26] I. Lenz, H. Lee, and A. Saxena, “Deep learning for detecting robotic grasps,” *The International Journal of Robotics Research*, vol. 34, no. 4-5, pp. 705–724, 2015.
- [27] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [28] Y. Bekiroglu, N. Marturi, M. A. Roa, K. J. M. Adjigble, T. Pardi, C. Grimm, R. Balasubramanian, K. Hang, and R. Stolkin, “Benchmarking protocol for grasp planning algorithms,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 315–322, 2019.

SUPPLEMENTARY MATERIALS

1) *Interval Score*: uses the prediction interval, with lower and upper endpoints represented by predictive quantiles at

level $\frac{\alpha}{2}$ and $1 - \frac{\alpha}{2}$. The Interval Score is defined as :

$$S_{\alpha}^{int}(l, u; y) = (u-l) + \frac{2}{\alpha}(l-y)\mathbf{1}\{y < l\} + \frac{2}{\alpha}(y-u)\mathbf{1}\{y > u\} \quad (4)$$

2) *Robot grasping experiment individual experiment* :
Experiment results of individual objects

TABLE IV: Result table from grasping experiment

	Generalized		Gaussian		Laplace		Cauchy		Evidential	
	P1	P2	P1	P2	P1	P2	P1	P2	P1	P2
Torch	9	5	6	2	8	5	5	0	4	0
Marker	9	4	7	1	7	2	5	1	5	1
Mouse	4	3	2	2	3	2	2	3	4	3
Brush	5	4	4	3	6	4	3	1	3	0
Stapler	6	7	1	4	3	5	1	3	1	2
Paste	8	8	4	4	8	7	5	5	4	3
Cube	6	6	5	5	6	6	5	5	5	5
Duster	6	5	4	2	7	5	2	1	3	0
Box	9	4	7	3	9	3	7	2	7	1
Allen	10	5	4	2	9	2	8	2	8	3

LIMITATIONS: The proposed loss function is not convex in general. To demonstrate non-convexity, consider the following scenario: Fixed α , Let $\alpha = 1$. Varying r , Choose two points $r1 = 1$ and $r2 = -1$. Convexity condition: A function f is convex if for all $r1, r2$ and $\lambda \in [0, 1]$:

$$f(\lambda x1 + (1 - \lambda)x2) \leq \lambda f(x1) + (1 - \lambda)L(x2)$$

Evaluating the loss function:

$$L(r1) = 1 - \log(\beta) + \log(\Gamma(1/\beta))$$

$$L(r2) = 1 - \log(\beta) + \log(\Gamma(1/\beta))$$

$$L(\lambda r1 + (1 - \lambda)r2) = |\lambda - (1 - \lambda)|^\beta - \log(\beta) + \log(\Gamma(1/\beta))$$

Checking the convexity condition: For $\lambda = 0.5$:

$$L(0.5r1 + 0.5r2) = 0 - \log(\beta) + \log(\Gamma(1/\beta))$$

$$\begin{aligned} &\lambda L(r1) + (1 - \lambda)L(r2) \\ &\quad 0.5(1 - \log(\beta) + \log(\Gamma(1/\beta))) \\ &+ 0.5(1 - \log(\beta) + \log(\Gamma(1/\beta))) = 1 - \log(\beta) + \log(\Gamma(1/\beta)) \end{aligned}$$

The convexity condition is not satisfied for all $\lambda \in [0, 1]$. Therefore, the loss function is not convex. While the loss function is not globally convex, it might be locally convex under certain conditions. However, the general form presented here does not guarantee convexity. We overcome this limitation by adding a software ode by clamping the loss to the very small number as possible in the cpu. This ensures that the residual never becomes zero and thus